

Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado

Aluno: Luis Eduardo Vieira de Oliveira

Quixadá-CE
2026

1. Introdução

1.1. Contexto: reuso de software e refatoração

Reuso é construir e evoluir software aproveitando o que já existe: componentes, bibliotecas, arquiteturas e padrões de projeto. Reduz custo e retrabalho e aumenta a confiabilidade. **Refatoração** é melhorar a estrutura interna do código sem alterar seu comportamento externo, reduzindo acoplamento e duplicação e aumentando a coesão.

As duas se conectam porque um código rígido, com comportamentos presos a hierarquias de herança ou a condicionais, dificulta o reuso. A refatoração com padrões GoF pode definir abstrações, separar responsabilidades e organizar variações de comportamento, tornando o código mais fácil de estender e reaproveitar.

1.2. Sistema selecionado e finalidade

O JPacman Framework é uma implementação em Java de um jogo no estilo Pac-Man, criada pelo grupo SERG da TU Delft para ensinar testes e engenharia de software. Tem interface gráfica, tabuleiro, níveis, movimentação do jogador e inteligência dos fantasmas, e usa Maven. A licença é Apache 2.0.

1.3. Motivação da escolha

A escolha do JPacman Framework foi motivada por sua base funcional, pela disponibilidade do código-fonte e pela existência de testes automatizados. O projeto tem finalidade didática e é utilizado como base para atividades em que estudantes modificam o código e aplicam conceitos de Engenharia de Software, incluindo padrões de projeto e testes. Nesse contexto, foram identificadas oportunidades de refatoração que permitem analisar como diferentes formas de organizar o código podem facilitar sua extensão, manutenção e reutilização.

1.4. Objetivo

O objetivo principal deste trabalho é refatorar componentes do JPacman Framework para facilitar possíveis extensões do sistema, preservando seu comportamento original. Busca-se estabelecer pontos de extensão que permitam introduzir novos comportamentos e operações aproveitando estruturas existentes, com menor necessidade de duplicar código ou modificar componentes centrais. Para isso, são aplicados os padrões Strategy, Observer, Template Method e Iterator, além de uma intervenção complementar com Visitor. A análise considera como as alterações favorecem a extensibilidade e quais benefícios e custos apresentam para a manutenção e o reuso.

1.5. Repositório

- Fork: https://github.com/LuisDallas1/jpacman-framework

- Original: https://github.com/SERG-Delft/jpacman-framework
- Versão original marcada pela tag v0-original.

1.6. Ambiente de desenvolvimento e requisitos de execução

O projeto foi compilado, testado e executado no IntelliJ IDEA Community, utilizando o Maven integrado à IDE e um JDK 11 baixado pelo próprio IntelliJ. Recomenda-se esse ambiente para reproduzir a execução.

Para configurar e executar o projeto:

1. Instalar o IntelliJ IDEA e clonar o repositório pela opção **File > New > Project from Version Control**. Como alternativa, abrir a pasta já clonada.
2. Nas configurações de estrutura do projeto, utilizar a opção **Add SDK > Download JDK** para baixar um JDK 11 e selecioná-lo como SDK do projeto. Conferir também sua seleção nas configurações do Maven.
3. Aguardar a importação do projeto e o carregamento das dependências pelo Maven.
4. Executar os testes pelo painel Maven, na opção **Lifecycle > test.**
5. Abrir **Launcher.java** e executar o método main pelo botão de execução da IDE.

2. Caracterização do sistema e situação inicial

2.1. Finalidade do Sistema

O **JPacman Framework** é uma aplicação interativa desenvolvida em Java pelo Software Engineering Research Group (SERG) da Delft University of Technology (TU Delft). Sua finalidade primária é didática e de pesquisa em Engenharia de Software, servindo como uma base de código realista para o estudo e aplicação de:

- Técnicas de refatoração e padrões de projeto orientados a objetos (GoF);
- Testes automatizados de software (TDD, BDD, testes unitários, testes de integração e end-to-end);
- Análise estática de código e controle de métricas de qualidade (cobertura, complexidade ciclomática e *code smells*).

2.2. Principais Funcionalidades

1. **Configuração e Carregamento de Tabuleiro:** Leitura e *parsing* matricial de mapas a partir de arquivos de texto formatados (ex.: board.txt), instanciando paredes, corredores transitáveis, pastilhas e posições iniciais.

2. **Ciclo de Vida da Partida:** Gerenciamento dos estados do jogo (iniciar, pausar, vitória e derrota) com notificações via padrão *Observer*.
3. **Controle do Jogador (Pac-Man):** Captura de comandos do teclado para movimentação nas quatro direções cardeais, controle de vidas e acumulação de pontos ao consumir pastilhas.
4. **Agendamento e Inteligência Artificial dos NPCs (Fantasmas):** Movimentação autônoma de quatro entidades não jogáveis com personalidades e algoritmos distintos de perseguição:
    - **Blinky (Shadow):** Persegue ativamente o Pac-Man calculando o caminho mais curto.
    - **Pinky (Speedy):** Tenta emboscar o Pac-Man mirando quatro posições à frente de sua direção atual.
    - **Inky (Bashful):** Utiliza um vetor relativo entre a posição de Blinky e a do jogador para calcular sua rota.
    - **Clyde (Pokey):** Persegue o jogador quando está distante, mas foge em direção oposta quando se aproxima a menos de 8 células.
5. **Sistema de Colisões e Interações:** Resolução de contatos entre unidades do tabuleiro (jogador versus pastilha gera pontuação; jogador versus fantasma resulta em morte do jogador).
6. **Interface Gráfica (GUI):** Renderização visual 2D construída em Java Swing com suporte a sprites animados e painéis informativos de pontuação e controles.

2.3. Linguagem, Frameworks e Tecnologias Utilizadas

- **Linguagem:** Java 11 (código-fonte compilado em bytecode padrão da JVM).
- **Build e Gerenciamento de Dependências:** Apache Maven (padronizando ciclo de vida de compilação, empacotamento em JAR executável via maven-shade-plugin e relatórios).
- **Ecossistema de Testes Automatizados:**
    - **JUnit 4 / JUnit Jupiter (5.9.3)** e junit-vintage-engine para execução de testes de unidade e suítes de regressão;
    - **Mockito (4.11.0)** para criação de dublês de teste (*mocks* e *stubs*);
    - **AssertJ (3.9.1)** para asserções fluentes;
    - **Cucumber JVM (1.2.5)** para testes de aceitação e cenários de comportamento BDD (*Behavior-Driven Development*).
- **Bibliotecas Utilitárias:** Google Guava (coleções imutáveis).
- **Ferramental de Qualidade e Análise Estática:**
    - **JaCoCo (0.8.12):** Medição de cobertura de testes com limite mínimo configurado;
    - **PMD, SpotBugs e Checkstyle:** Análise estática de vulnerabilidades, convenções de código e métricas de complexidade.
- **Estrutura Geral do Projeto:**

```
nl.tudelft.jpacman/
├── board/          # Modelo de grade espacial (Board, Square, Unit, Direction, BoardFactory)
├── game/           # Orquestração do ciclo de jogo (Game, SinglePlayerGame, GameFactory)
├── level/          # Regras da fase (Level, Player, Pellet, MapParser, CollisionMap)
├── npc/            # Abstrações base para personagens autônomos (Ghost)
│   └── ghost/      # Implementações dos fantasmas (Blinky, Pinky, Inky, Clyde, Navigation, GhostFactory)
├── sprite/         # Manipulação de imagens e animações (Sprite, PacManSprites, AnimatedSprite)
├── ui/             # Camada visual Swing (PacManUI, PacManUiBuilder, BoardPanel, ScorePanel)
└── Launcher.java   # Ponto de entrada (Bootstrap, orquestração e instanciação de fábricas)
```

3. Problemas Identificados e Refatorações Realizadas

Os padrões selecionados, Strategy, Observer, Template Method, Iterator e Visitor, pertencem à categoria dos padrões comportamentais GoF, pois organizam algoritmos, responsabilidades e formas de comunicação entre objetos.

3.1. Strategy: comportamento dos fantasmas

3.1.1. Problema

Natureza: dependência de herança para variar comportamento e duplicação de código. O problema é a rigidez identificada nesse caso; utilizar herança não constitui, por si só, uma violação.

Descrição: a classe abstrata Ghost exige que cada fantasma implemente seu algoritmo de movimento sobrescrevendo nextAiMove(). Para criar qualquer comportamento novo é necessário criar uma subclasse inteira de Ghost, e o comportamento fica fixado na criação do objeto. Além disso, as subclasses repetem a mesma sequência de passos: localizar o jogador, validar o alvo, calcular a rota mais curta e extrair o primeiro passo. A única diferença real entre elas é como o destino é escolhido.

Evidências:

Blinky.java:

```
Unit nearest = Navigation.findNearest(Player.class, getSquare());
if (nearest == null) {
    return Optional.empty();
}
assert nearest.hasSquare();
Square target = nearest.getSquare();

List<Direction> path = Navigation.shortestPath(getSquare(), target, this);
if (path != null && !path.isEmpty()) {
    return Optional.ofNullable(path.get(0));
}
return Optional.empty();
```

Pinky.java (mesma estrutura, mudando apenas a escolha do destino):

```
Unit player = Navigation.findNearest(Player.class, getSquare());
if (player == null) {
    return Optional.empty();
}
assert player.hasSquare();
Square destination = player.squaresAheadOf(SQUARES_AHEAD);

List<Direction> path = Navigation.shortestPath(getSquare(), destination, this);
if (path != null && !path.isEmpty()) {
    return Optional.ofNullable(path.get(0));
}
return Optional.empty();
```

Em LevelFactory.java, para ter um fantasma que apenas vaga aleatoriamente, foi preciso criar uma classe interna cujo único papel é devolver Optional.empty():

```
private static final class RandomGhost extends Ghost {
    RandomGhost(Map<Direction, Sprite> ghostSprite) {
        super(ghostSprite, (int) DELAY, 0);
    }
    @Override
    public Optional<Direction> nextAiMove() {
        return Optional.empty(); // Força o Ghost.nextMove() a usar randomMove()
    }
}
```

Impacto: o comportamento de um fantasma não pode ser trocado durante a partida, os algoritmos de movimento não podem ser reaproveitados fora da hierarquia de Ghost, e qualquer correção na rotina de rota precisa ser replicada em vários arquivos.

Padrão escolhido: Strategy. A escolha do destino vira uma estratégia injetada no Ghost, e o esqueleto comum fica em um único lugar.

3.1.2. Refatoração

12 arquivos foram adicionados/modificados:

Arquivo	Papel GoF
Ghost	Context
GhostTargetStrategy	Strategy
BlinkyTargetStrategy	ConcreteStrategy
PinkyTargetStrategy	ConcreteStrategy
ClydeTargetStrategy	ConcreteStrategy
InkyTargetStrategy	ConcreteStrategy
RandomTargetStrategy	ConcreteStrategy
Blinky, Pinky, Inky, Clyde	Especialização do Context / configuração
LevelFactory	Client

Principais ganhos

- **Extensibilidade:** novas formas de escolher o destino dos fantasmas podem ser implementadas por meio de GhostTargetStrategy, sem alterar a rotina comum de movimentação. A estratégia também pode ser substituída durante a partida, sem recriar o fantasma.
- **Reutilização e redução de duplicação:** a rotina de busca do caminho e obtenção do próximo movimento fica centralizada em Ghost, sendo compartilhada pelas diferentes estratégias de destino.
- **Separação de responsabilidades:** Ghost controla a movimentação, enquanto as estratégias definem a escolha do destino de acordo com cada comportamento.
- **Simplificação da configuração:** o comportamento aleatório passa a ser configurado com RandomTargetStrategy, dispensando a classe interna RandomGhost.

Trechos relevantes da implementação:

Antes:

```java
// Blinky.java — ANTES
@Override
public Optional<Direction> nextAiMove() {
    assert hasSquare();

    Unit nearest = Navigation.findNearest(Player.class, getSquare());
    if (nearest == null) {
        return Optional.empty();
    }
    assert nearest.hasSquare();
    Square target = nearest.getSquare();        // ← diferença real

    // ↓ passos idênticos em Pinky, Clyde, Inky
    List<Direction> path = Navigation.shortestPath(getSquare(), target, this);
    if (path != null && !path.isEmpty()) {
        return Optional.ofNullable(path.get(0));
    }
    return Optional.empty();
}
```

```java
// Pinky.java — ANTES (mesma estrutura, só o destino muda)
@Override
public Optional<Direction> nextAiMove() {
    assert hasSquare();

    Unit player = Navigation.findNearest(Player.class, getSquare());
    if (player == null) {
        return Optional.empty();
    }
    assert player.hasSquare();
    Square destination = player.squaresAheadOf(SQUARES_AHEAD); // ← diferença real

    // ↓ passos idênticos em Blinky, Clyde, Inky
    List<Direction> path = Navigation.shortestPath(getSquare(), destination, this);
    if (path != null && !path.isEmpty()) {
        return Optional.ofNullable(path.get(0));
    }
    return Optional.empty();
}
```

Depois:

```java
// GhostTargetStrategy.java — interface Strategy
public interface GhostTargetStrategy {
    Optional<Square> selectTarget(Unit ghost);
}
```

java

```java
// BlinkyTargetStrategy.java — ConcreteStrategy
public class BlinkyTargetStrategy implements GhostTargetStrategy {
    @Override
    public Optional<Square> selectTarget(Unit ghost) {
        Unit nearest = Navigation.findNearest(Player.class, ghost.getSquare());
        if (nearest == null) return Optional.empty();
        return Optional.of(nearest.getSquare());
    }
}
```

java

```java
// PinkyTargetStrategy.java — ConcreteStrategy
public class PinkyTargetStrategy implements GhostTargetStrategy {
    private static final int SQUARES_AHEAD = 4;

    @Override
    public Optional<Square> selectTarget(Unit ghost) {
        Unit player = Navigation.findNearest(Player.class, ghost.getSquare());
        if (player == null) return Optional.empty();
        return Optional.of(player.squaresAheadOf(SQUARES_AHEAD));
    }
}
```

3.2. Observer: notificação de eventos do jogador

3.2.1. Problema

Natureza: polling contínuo (pull em vez de push), modelo sem mecanismo de notificação e alto acoplamento entre a interface e o estado interno de Player.
Descrição: a interface gráfica não é notificada quando o estado do jogador muda (ganho de pontuação ou morte). Em vez de o modelo avisar os interessados, o PacManUI mantém uma tarefa periódica a cada 40 ms (25 vezes por segundo) que chama scorePanel.refresh(). O ScorePanel consulta ativamente cada jogador (player.getScore(), player.isAlive()) e reescreve os rótulos, mesmo quando nada mudou. Já a classe Player altera seu estado sem oferecer nenhum meio de os observadores externos reagirem.

Evidências:

PacManUI.java (tarefa periódica que força a consulta do placar):

```
public void start() {
    setVisible(true);
    ScheduledExecutorService service = Executors.newSingleThreadScheduledExecutor();
    service.scheduleAtFixedRate(this::nextFrame, 0, FRAME_INTERVAL, TimeUnit.MILLISECONDS);
}

private void nextFrame() {
    boardPanel.repaint();
    scorePanel.refresh(); // Chamado a cada 40ms mesmo se a pontuação não mudou
}
```

ScorePanel.java (busca ativa dos dados e reescrita dos rótulos):

```
protected void refresh() {
    for (Map.Entry<Player, JLabel> entry : scoreLabels.entrySet()) {
        Player player = entry.getKey();
        String score = "";
        if (!player.isAlive()) {
            score = "You died. ";
        }
        score += scoreFormatter.format(player);
        entry.getValue().setText(score);
    }
}
```

Player.java (mudanças de estado sem nenhuma notificação):

```
public void setAlive(boolean isAlive) {
    if (isAlive) {
        deathSprite.setAnimating(false);
    }
    if (!isAlive) {
        deathSprite.restart();
    }
    this.alive = isAlive; // o jogador more e nenhum componente externo é avisado
}

public void addPoints(int points) {
    score += points; // pontuação acumulada sem notificar ninguém
}
```

Precedente no próprio sistema: o Level já adota o padrão Observer para eventos do nível, por meio da interface LevelObserver, que notifica interessados quando o nível é vencido ou

perdido. Os eventos do jogador, no entanto, não seguem o mesmo modelo: Player não oferece nenhum mecanismo de notificação, e a interface depende de polling para refletir mudanças de pontuação e de vida.

Impacto:

1. **Sincronização dependente de temporizador:** a consistência entre modelo e tela depende de um loop periódico, o que dificulta testar o ScorePanel isoladamente, já que ele precisa ser chamado manualmente em vez de reagir a eventos.
2. **Baixa extensibilidade:** para tocar um som ao coletar pastilhas, registrar estatísticas ou exibir conquistas por pontuação, não há ponto de ligação. Seria preciso alterar nextFrame() ou a lógica de colisão do domínio.
3. **Trabalho redundante (secundário):** o placar é reformatado 25 vezes por segundo sem mudança. O custo é baixo, mas o trabalho é desnecessário.

Padrão escolhido: Observer. Player passa a ser o *subject* e notifica PlayerObservers (por exemplo, scoreChanged, playerDied). O ScorePanel se registra como observador e atualiza o rótulo só quando há evento, e novos interessados (som, estatísticas) entram sem alterar Player nem a UI.

3.2.2. Refatoração

4 arquivos foram adicionados/modificados:

Arquivo	Papel GoF
PlayerObserver	ObserverInterface
Player	Subject
ScorePanel	ConcreteObserver
PacManUI	Client

Principais ganhos

- **Extensibilidade:** novos componentes, como efeitos sonoros, estatísticas ou conquistas, podem reagir aos eventos do jogador implementando PlayerObserver e registrando-se como observadores.
- **Reutilização do mecanismo de notificação:** diferentes observadores podem utilizar os mesmos eventos de pontuação e morte, sem exigir mecanismos específicos de comunicação para cada componente.
- **Separação de responsabilidades:** Player comunica as mudanças de estado, enquanto cada observador define como reagir a elas. O ScorePanel fica responsável pela atualização visual do placar**.**

- **Atualização orientada a eventos:** o placar passa a ser atualizado em resposta às notificações, dispensando sua consulta e reescrita periódica pelo temporizador.
- **Foram adicionados 2 arquivos de testes.**

Trechos relevantes da implementação:

Antes:

Player:

```
private boolean alive;

public void setAlive(boolean isAlive) {
    if (isAlive) {
        deathSprite.setAnimating(false);
    }
    if (!isAlive) {
        deathSprite.restart();
    }
    this.alive = isAlive;          // muda de estado, mas ninguém é avisado
}

public void addPoints(int points) {
    score += points;               // idem: nenhuma notificação
}
```

PacManUI:

```
private void nextFrame() {
    boardPanel.repaint();
    scorePanel.refresh();          // empurra a cada 40 ms (25x/s)
}
```

Depois:

Player:

```
private final Set<PlayerObserver> observers = new HashSet<>();

public void addObserver(PlayerObserver observer)    { observers.add(observer); }
public void removeObserver(PlayerObserver observer) { observers.remove(observer); }

public void setAlive(boolean isAlive) {
    this.alive = isAlive;          // estado atualizado ANTES de notificar
    if (isAlive) {
        deathSprite.setAnimating(false);
    } else {
        deathSprite.restart();
        notifyPlayerDied();        // push para os observadores
    }
}

public void addPoints(int points) {
    score += points;
    notifyScoreChanged();          // push para os observadores
}

private void notifyScoreChanged() {
    for (PlayerObserver observer : observers) {
        observer.scoreChanged(score);
    }
}
```

PlayerObserver:

```
public interface PlayerObserver {
    void scoreChanged(int newScore);
    void playerDied();
}
```

PacManUI:

```
private void nextFrame() {
    boardPanel.repaint();          // só o tabuleiro; placar reage a eventos
}
```

3.3. Template Method: algoritmo de busca em largura

3.3.1. Problema

Natureza: duplicação de código algorítmico e falta de abstração para a travessia da malha de células.

Descrição: a classe utilitária Navigation concentra as rotinas de busca usadas pela IA dos NPCs. Os métodos shortestPath (caminho mais curto até uma célula) e findNearest (unidade mais próxima de determinado tipo) implementam o mesmo esqueleto de busca em largura sobre a malha de Squares: inicializam uma fila e um conjunto visited, retiram o primeiro elemento, testam a condição de parada, marcam a célula como visitada e expandem os quatro vizinhos de Direction.values(). A variação se resume a três pontos: a condição de parada (alcançar uma célula versus encontrar uma unidade de um tipo), o filtro de vizinhos e o resultado devolvido. Como o esqueleto não está isolado, cada método mantém sua própria cópia, e as cópias já divergem em detalhes.

Evidências:

Navigation.java, esqueleto de BFS em shortestPath:

```
List<Node> targets = new ArrayList<>();
Set<Square> visited = new HashSet<>();
targets.add(new Node(null, from, null));
while (!targets.isEmpty()) {
    Node node = targets.remove(0);
    Square square = node.getSquare();
    if (square.equals(to)) {
        return node.getPath();
    }
    visited.add(square);
    addNewTargets(traveller, targets, visited, node, square);
}
return null;
```

Navigation.java, filtro de vizinhos de shortestPath (considera acessibilidade):

```
for (Direction direction : Direction.values()) {
    Square newTarget = square.getSquareAt(direction);
    if (!visited.contains(newTarget) && !toDo.contains(newTarget)) {
        toDo.add(newTarget);
    }
}
```

Navigation.java, mesmo esqueleto em findNearest:

```
List<Square> toDo = new ArrayList<>();
Set<Square> visited = new HashSet<>();
toDo.add(currentLocation);
while (!toDo.isEmpty()) {
    Square square = toDo.remove(0);
    Unit unit = findUnit(type, square);
    if (unit != null) {
        assert unit.hasSquare();
        return unit;
    }
    visited.add(square);
    for (Direction direction : Direction.values()) {
        Square newTarget = square.getSquareAt(direction);
        if (!visited.contains(newTarget) && !toDo.contains(newTarget)) {
            toDo.add(newTarget);
        }
    }
}
return null;
```

Divergências entre as cópias: shortestPath considera a acessibilidade das células quando recebe um viajante, enquanto findNearest não considera (comportamento equivalente ao modo traveller == null do primeiro, mas não documentado no Javadoc de findNearest). Além disso, findNearest evita enfileirar células repetidas com toDo.contains(...), enquanto shortestPath não evita. Ambos usam ArrayList como fila com remove(0).

Impacto:

1. **Correções replicadas:** qualquer ajuste no algoritmo (por exemplo, trocar a fila por uma estrutura mais eficiente) precisa ser feito e testado em dois lugares.
2. **Divergência entre cópias:** as diferenças acima mostram que as implementações evoluíram separadas, e não há um ponto único que defina como a busca funciona.
3. **Extensão custosa:** uma nova busca sobre a malha (outro critério de parada, por exemplo) tende a ganhar uma terceira cópia, pois a lógica fica presa em métodos estáticos sem ponto de reaproveitamento.

Padrão escolhido: Template Method. Cria-se uma classe base abstrata BreadthFirstSearch<T> com um método-gabarito invariante execute(Square start). As variações (isGoal, canTraverse e extractResult) viram métodos gancho implementados por subclasses especializadas (ShortestPathSearch e NearestUnitSearch), e os métodos estáticos de Navigation passam a delegar a elas, mantendo a API pública.

3.3.2. Refatoração

4 arquivos foram adicionados/modificados:

Arquivo	Papel GoF
BreadthFirstSearch	AbstractClass
ShortestPathSearch	ConcreteClass
NearestUnitSearch	ConcreteClass
Navigation	Client/Facade

Principais ganhos

- **Extensibilidade:** novas buscas podem ser implementadas por subclasses de BreadthFirstSearch, definindo a condição de parada, as regras de travessia e a extração do resultado.
- **Reutilização e redução de duplicação:** as buscas compartilham o mesmo esqueleto de BFS, incluindo o gerenciamento da fila e das células visitadas. Correções nessa rotina ficam concentradas em uma única implementação.
- **Separação de responsabilidades:** a classe base controla a execução da busca, enquanto as subclasses definem suas variações específicas.
- **Organização da travessia:** ArrayDeque substitui a fila baseada em ArrayList.remove(0), e a marcação das células ao enfileirá-las evita entradas repetidas. Essas alterações não tiveram seu impacto no tempo de execução medido.
- **Compatibilidade com os clientes:** Navigation.shortestPath e Navigation.findNearest mantêm suas assinaturas, permitindo utilizar a estrutura refatorada pela mesma API.

Trechos relevantes da implementação:

Antes — esqueleto de BFS em `Navigation.shortestPath`:

```java
List<Node> targets = new ArrayList<>();
Set<Square> visited = new HashSet<>();
targets.add(new Node(null, from, null));

while (!targets.isEmpty()) {
    Node node = targets.remove(0);
    Square square = node.getSquare();

    if (square.equals(to)) {
        return node.getPath();
    }

    visited.add(square);
    addNewTargets(traveller, targets, visited, node, square);
}
```

Antes — outro esqueleto de BFS em `Navigation.findNearest`:

Java

```java
while (!toDo.isEmpty()) {
    Square square = toDo.remove(0);
    Unit unit = findUnit(type, square);

    if (unit != null) {
        assert unit.hasSquare();
        return unit;
    }

    visited.add(square);
    for (Direction direction : Direction.values()) {
        Square newTarget = square.getSquareAt(direction);
        if (!visited.contains(newTarget)
            && !toDo.contains(newTarget)) {
            toDo.add(newTarget);
        }
    }
}
```

Depois — trecho do método-gabarito em `BreadthFirstSearch.execute`:

Java

```java
while (!queue.isEmpty()) {
    Square current = queue.pollFirst();

    if (isGoal(current)) {
        return extractResult(current, parentMap, directionToReach);
    }

    for (Direction direction : Direction.values()) {
        Square neighbor = current.getSquareAt(direction);

        if (neighbor == null) {
            continue;
        }
        if (visited.contains(neighbor)) {
            continue;
        }
        if (!canTraverse(current, direction, neighbor)) {
            continue;
        }

        visited.add(neighbor);
        parentMap.put(neighbor, current);
        directionToReach.put(neighbor, direction);
        queue.addLast(neighbor);
    }
}
```

Depois — especialização em `ShortestPathSearch`:

```java
@Override
protected boolean isGoal(Square current) {
    return current.equals(target);
}

@Override
protected boolean canTraverse(
    Square current, Direction direction, Square neighbor) {

    if (traveller != null) {
        return neighbor.isAccessibleTo(traveller);
    }
    return true;
}
```

3.4. Iterator: travessia das células do tabuleiro

3.4.1. Problema

Natureza: duplicação de código (laços de varredura repetidos), acoplamento dos clientes à indexação da grade e ausência de abstração de travessia.

Descrição: Board representa o tabuleiro por uma matriz Square[][] privada, mas não implementa Iterable nem oferece qualquer mecanismo de iteração. Por isso, os subsistemas que precisam inspecionar as células reproduzem laços aninhados sobre getWidth() e getHeight(), com estrutura repetida no domínio (Level.java), na renderização (BoardPanel.java) e na navegação (Navigation.java). Além da repetição, a ordem de varredura não é uniforme: Level percorre por coluna (x depois y), enquanto BoardPanel e Navigation percorrem por linha (y depois x).

Evidências:
Level.java (contagem de pastilhas restantes):

```java
public int remainingPellets() {
    Board board = getBoard();
    int pellets = 0;
    for (int x = 0; x < board.getWidth(); x++) {
        for (int y = 0; y < board.getHeight(); y++) {
            for (Unit unit : board.squareAt(x, y).getOccupants()) {
                if (unit instanceof Pellet) {
                    pellets++;
                }
            }
        }
    }
    return pellets;
}
```

BoardPanel.java (desenho do tabuleiro):

```java
for (int y = 0; y < board.getHeight(); y++) {
    for (int x = 0; x < board.getWidth(); x++) {
        int cellX = x * cellW;
        int cellY = y * cellH;
        Square square = board.squareAt(x, y);
        render(square, graphics, cellX, cellY, cellW, cellH);
    }
}
```

Navigation.java (localização de uma unidade no tabuleiro):

```java
public static <T extends Unit> T findUnitInBoard(Class<T> clazz, Board board) {
    for (int y = 0; y < board.getHeight(); y++) {
        for (int x = 0; x < board.getWidth(); x++) {
            final T ghost = Navigation.findUnit(clazz, board.squareAt(x, y));
            if (ghost != null) {
                return ghost;
            }
        }
    }
    return null;
}
```

Impacto:

- **Acoplamento à indexação da grade:** cada cliente depende de x, y, getWidth() e getHeight() para percorrer as células. Mudar a forma de percorrer (ordem, filtros, representação interna) exige alterar todos os clientes.
- **Duplicação de código de travessia:** três pacotes diferentes (level, ui, npc.ghost) mantêm sua própria varredura.

- **Menos reaproveitamento:** sem uma abstração de iteração, não é possível usar construções como o *enhanced for* sobre o tabuleiro, nem compor a travessia com outras operações de forma simples.

Padrão escolhido: Iterator. Board passa a ser a coleção iterável e um iterador dedicado encapsula a travessia das células, de modo que os clientes percorrem o tabuleiro sem conhecer a indexação.

3.4.2. Refatoração

5 arquivos foram adicionados/modificados:

Arquivo	Papel GoF
Board	ConcreteAggregate
BoardIterator	ConcreteIterator
Level	Client
Navigation	Client
BoardPanel	Client

Principais ganhos

- **Extensibilidade:** novas operações de contagem, verificação ou inspeção podem percorrer o tabuleiro pela interface de iteração, sem implementar outra varredura bidimensional.
- **Reutilização e redução de duplicação:** o avanço entre as células fica centralizado em BoardIterator, compartilhando o mecanismo de travessia entre os clientes.
- **Separação de responsabilidades:** o iterador define como percorrer o tabuleiro, enquanto cada cliente define a operação realizada sobre as células.
- **Menor dependência da indexação:** as operações de contagem e localização deixam de controlar coordenadas e dimensões para percorrer o tabuleiro. A renderização ainda pode utilizar coordenadas para posicionar os elementos na tela.
- **Ordem de travessia definida:** a iteração por linha preserva o critério utilizado por Navigation para retornar a primeira unidade encontrada.
- **Foram adicionados 3 arquivos de testes.**

Trechos relevantes da implementação:

Antes — contagem de pastilhas em `Level`:

```java
for (int x = 0; x < board.getWidth(); x++) {
    for (int y = 0; y < board.getHeight(); y++) {
        for (Unit unit : board.squareAt(x, y).getOccupants()) {
            if (unit instanceof Pellet) {
                pellets++;
            }
        }
    }
}
```

Depois — contagem utilizando o iterador:

```java
for (Square square : board) {
    for (Unit unit : square.getOccupants()) {
        if (unit instanceof Pellet) {
            pellets++;
        }
    }
}
```

Depois — iterador dedicado:

Java

```java
final class BoardIterator implements Iterator<Square> {

    private final Board board;
    private int x;
    private int y;

    BoardIterator(Board board) {
        this.board = board;
    }

    @Override
    public boolean hasNext() {
        return board.getWidth() > 0
            && y < board.getHeight();
    }

    @Override
    public Square next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }

        Square square = board.squareAt(x, y);
        x++;

        if (x == board.getWidth()) {
            x = 0;
            y++;
        }

        return square;
    }
}
```

3.5. Visitor: resolução de colisões (complementar)

3.5.1. Problema

Natureza: code smell (verificação de tipos com instanceof e downcasting), alto acoplamento a classes concretas e dificuldade de extensão (Open/Closed).

Descrição: PlayerCollisions resolve a colisão entre duas unidades inspecionando suas classes concretas em cascatas de instanceof seguidas de cast. Como Java só faz *single dispatch* (o método chamado depende do tipo em tempo de execução do receptor, não dos

argumentos), a classe simula o despacho múltiplo de forma procedural. Qualquer nova unidade interativa (por exemplo, PowerPellet, Fruit ou Door) obriga a reabrir e modificar PlayerCollisions.

Evidências:

```java
@Override
public void collide(Unit mover, Unit collidedOn) {
    if (mover instanceof Player) {
        playerColliding((Player) mover, collidedOn);
    }
    else if (mover instanceof Ghost) {
        ghostColliding((Ghost) mover, collidedOn);
    }
    else if (mover instanceof Pellet) {
        pelletColliding((Pellet) mover, collidedOn);
    }
}

private void playerColliding(Player player, Unit collidedOn) {
    if (collidedOn instanceof Ghost) {
        playerVersusGhost(player, (Ghost) collidedOn);
    }
    if (collidedOn instanceof Pellet) {
        playerVersusPellet(player, (Pellet) collidedOn);
    }
}
```

PlayerCollisions.java (Javadoc da própria classe):

```java
/**
 * A simple implementation of a collision map for the JPacman player.
 * <p>
 * It uses a number of instanceof checks to implement the multiple dispatch for the
 * collisionmap. For more realistic collision maps, this approach will not scale,
 * and the recommended approach is to use a {@link CollisionInteractionMap}.
 */
```

LevelFactory.java (a versão com instanceof é a usada por padrão):

```java
// We'll adopt the simple collision map for now.
CollisionMap collisionMap = new PlayerCollisions();
```

Solução já existente no repositório: devido ao propósito didático, o projeto inclui o CollisionInteractionMap (usado por DefaultPlayerInteractionMap), que resolve o problema de extensão registrando handlers por par de tipos, sem editar PlayerCollisions. Em contrapartida, ele descobre a classe mais específica por reflexão (getClass, getSuperclass,

getInterfaces), usa cast não verificado (@SuppressWarnings("unchecked")) e ignora silenciosamente uma colisão para a qual não haja handler registrado.

Impacto:

- **Extensão:** cada nova unidade exige editar a cadeia de condicionais de PlayerCollisions.
- **Verificação só em execução:** uma combinação de tipos não prevista é ignorada em silêncio, sem erro de compilação.
- **Mistura de responsabilidades:** a lógica de despacho fica no mesmo lugar das regras de colisão.

Padrão aplicável: Visitor (double dispatch). As unidades aceitam um visitante, e a resolução das colisões utiliza despacho duplo, com chamadas polimórficas e assinaturas verificadas pelo compilador, sem instanceof, sem cast e sem reflexão. Adicionar um novo tipo de unidade ainda exige um novo método visit e um accept na unidade. A alteração do contrato exige que as implementações forneçam os métodos previstos, permitindo verificar suas assinaturas em compilação, mas sem garantir a correção das regras implementadas.

Tratamento: registrado como problema complementar (apesar de também implementado). Como o repositório já traz uma alternativa funcional para este problema (o CollisionInteractionMap), ele não é contado entre os padrões principais deste trabalho.

3.5.2. Refatoração

7 arquivos foram adicionados/modificados:

Arquivo	Papel GoF
UnitVisitor	Visitor
Unit	Element
Player	ConcreteElement
Ghost	ConcreteElement
Pellet	ConcreteElement
PlayerCollisions	Coordenador do despacho
LevelFactory	Client

Principais ganhos

- **Extensibilidade de operações:** novos visitantes podem acrescentar operações sobre os tipos de unidade existentes, aproveitando o contrato de visita. A inclusão de novos

tipos de unidade, entretanto, pode exigir alterações nesse contrato e em suas implementações.

- **Reutilização do mecanismo de visita:** diferentes operações podem utilizar o mesmo mecanismo de despacho, sem reproduzir cadeias de verificação explícita de tipos.
- **Separação de responsabilidades:** o mecanismo de visita organiza o despacho entre os tipos, enquanto as operações expressam as regras de colisão.
- **Contratos tipados:** as operações recebem unidades com tipos definidos, permitindo a verificação das assinaturas pelo compilador e dispensando instanceof, downcasting e reflexão na resolução refatorada.
- **Tratamento explícito**: as combinações sem efeito podem ser representadas por operações intencionalmente vazias. A existência de um método no contrato, contudo, não garante a correção da regra implementada.
- **Foram adicionados 2 arquivos de testes.**

Trechos relevantes da implementação:

Antes — despacho procedural em `PlayerCollisions`:

Java

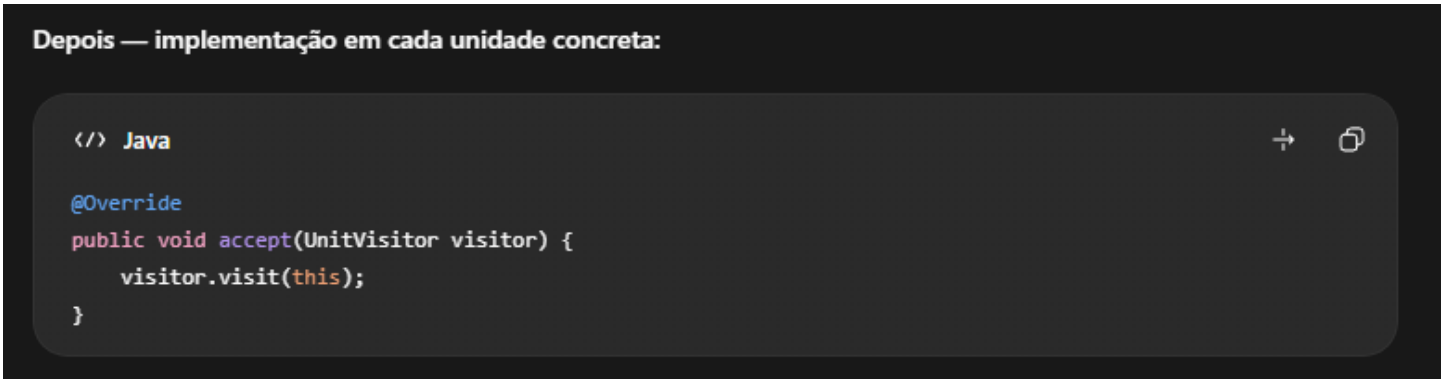
```java
@Override
public void collide(Unit mover, Unit collidedOn) {
    if (mover instanceof Player) {
        playerColliding((Player) mover, collidedOn);
    }
    else if (mover instanceof Ghost) {
        ghostColliding((Ghost) mover, collidedOn);
    }
    else if (mover instanceof Pellet) {
        pelletColliding((Pellet) mover, collidedOn);
    }
}
```

Depois — contrato de visita proposto:

Java

```java
public interface UnitVisitor {
    void visit(Player player);
    void visit(Ghost ghost);
    void visit(Pellet pellet);
}
```

Depois — implementação em cada unidade concreta:



4. Análise dos Resultados

As refatorações ampliaram as possibilidades de extensão do JPacman Framework ao criar pontos específicos para introduzir novos comportamentos e operações. O principal benefício prático está na possibilidade de evoluir o sistema aproveitando estruturas existentes, sem repetir algoritmos ou concentrar novas regras nas mesmas classes.

4.1. Extensibilidade e reutilização

Com Strategy, a escolha do destino dos fantasmas passou a ser definida por estratégias independentes da rotina comum de movimentação. Assim, comportamentos como perseguir outro alvo, fugir para uma região do tabuleiro ou alternar entre perseguição e movimentação aleatória podem ser introduzidos por novas implementações de GhostTargetStrategy. A possibilidade de substituir a estratégia durante a partida também permite combinar comportamentos conforme as condições do jogo, sem recriar o fantasma.

Observer criou um ponto de extensão para componentes interessados nas mudanças do jogador. Além do placar, funcionalidades como efeitos sonoros, registro de estatísticas e conquistas podem reagir às notificações de pontuação e morte por meio de novos observadores. Dessa forma, essas funcionalidades podem ser acrescentadas sem inserir suas regras diretamente em Player ou no temporizador da interface.

Template Method permite implementar novas buscas sobre o tabuleiro reutilizando o esqueleto de BFS. Uma busca pela pastilha mais próxima ou por uma célula que satisfaça determinado critério, por exemplo, pode especializar a classe base e definir suas condições de parada, travessia e extração do resultado. O gerenciamento da fila e das células visitadas permanece compartilhado, reduzindo o trabalho necessário para introduzir outra busca.

Iterator facilita a inclusão de operações que precisam percorrer o tabuleiro. Contagens, verificações de estado e outras inspeções podem utilizar a mesma interface de iteração, sem reproduzir laços bidimensionais. O reuso ocorre na própria travessia: cada cliente define o que fazer com as células, enquanto o iterador determina como percorrê-las. Na resolução de colisões, Visitor organiza as operações sobre os tipos de unidades existentes e torna explícitos os tratamentos previstos. Esse benefício tem um limite: acrescentar um novo tipo de unidade pode exigir alterações na interface do

visitante e em suas implementações. Portanto, sua contribuição para a extensibilidade depende de quais partes do sistema variam com maior frequência.

Esses exemplos representam possibilidades oferecidas pela estrutura refatorada, e não funcionalidades adicionais já implementadas. O ganho está em disponibilizar caminhos definidos para futuras extensões, aproveitando algoritmos, interfaces e mecanismos comuns.

4.2. Acoplamento e organização das responsabilidades

Os pontos de extensão foram obtidos pela separação entre operações compartilhadas e decisões específicas. Ghost executa a rotina comum de movimentação e delega a escolha do destino; Player informa mudanças de estado e os observadores decidem como reagir; BreadthFirstSearch controla a busca e suas subclasses definem as variações; BoardIterator conduz a travessia e os clientes realizam suas operações.

Essa organização reduz a necessidade de modificar componentes centrais ao acrescentar funcionalidades que utilizem os pontos de extensão criados. As dependências continuam existindo, mas passam a ser expressas por contratos mais delimitados, facilitando a compreensão das responsabilidades e a substituição de implementações.

4.3. Duplicação e manutenção

A centralização das rotinas de movimentação, busca e travessia reduz a quantidade de lugares em que uma mesma correção precisa ser realizada. Ajustes no esqueleto de BFS, por exemplo, passam a atender às buscas que o reutilizam, enquanto mudanças no percurso das células ficam concentradas no iterador.

Na busca em largura, a utilização de ArrayDeque e a marcação das células ao enfileirá-las também melhoram a organização do algoritmo, evitando o deslocamento de elementos de ArrayList.remove(0) e entradas repetidas na fila. Entretanto, o impacto dessas alterações no tempo de execução não foi medido.

A preservação das assinaturas de Navigation.shortestPath e Navigation.findNearest permite que seus clientes continuem utilizando a mesma API. Isso demonstra que a reorganização interna pode criar novas possibilidades de reuso sem exigir mudanças em todos os componentes consumidores.

4.4. Validação das refatorações

A validação teve como objetivo verificar se os comportamentos exercitados continuaram funcionando após a reorganização do código. Antes das modificações, a execução da suíte apresentou 45 testes passando, estabelecendo uma referência inicial para o sistema.

Após as refatorações, a execução pelo Maven apresentou 59 testes, com 0 falhas, 0 erros e 0 testes ignorados, encerrando com BUILD SUCCESS. Além dos testes pré-existentes, foram adicionados 14 testes, relacionados às notificações de mudanças no jogador, à iteração do tabuleiro e à resolução de colisões.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 59, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.652 s
[INFO] Finished at: 2026-10-13T11:41:34-03:00
[INFO] ------------------------------------------------------------------------
```

Resultado da execução dos testes automatizados após as refatorações.

As verificações manuais complementaram os testes automatizados pela observação do funcionamento integrado do jogo. Foram executados os cenários apresentados na tabela seguinte, comparando o comportamento observado com o esperado.

Cenário	Resultado esperado	Resultado observado
Iniciar uma partida	Tabuleiro, jogador e fantasmas são exibidos.	Conforme esperado
Movimentar o jogador	O jogador se movimenta e respeita as paredes.	Conforme esperado
Coletar pastilhas	A pastilha desaparece e a pontuação é atualizada no placar.	Conforme esperado
Observar os fantasmas	Os fantasmas se movimentam durante a partida.	Conforme esperado
Colidir com um fantasma	O jogador morre e o jogo apresenta a consequência prevista.	Conforme esperado
Pausar e retomar	A movimentação é interrompida e retomada.	Conforme esperado

Os resultados fornecem evidências de preservação dos comportamentos cobertos pelos testes e pelas verificações manuais. Entretanto, a validação não abrange necessariamente todos os cenários de uso, e a ausência de falhas nas execuções realizadas não garante a inexistência de defeitos.

4.5. Comparação antes e depois

Aspecto	Situação anterior	Situação após as refatorações
Duplicação	Rotinas comuns de movimentação, busca e travessia eram repetidas em diferentes componentes.	A rotina de movimentação foi centralizada em Ghost, o esqueleto de BFS em BreadthFirstSearch e a travessia em BoardIterator.
Acoplamento	A escolha do comportamento dependia das subclasses de Ghost, e clientes do tabuleiro reproduziam sua indexação.	A escolha do destino passou a depender de GhostTargetStrategy, e as operações de contagem e localização passaram a utilizar a interface de iteração.
Coesão	Decisões específicas e operações compartilhadas apareciam na mesma implementação.	Estratégias definem destinos, observadores reagem a eventos e subclasses de busca definem variações do algoritmo.
Extensibilidade	Novos comportamentos ou operações podiam exigir alterações em classes existentes ou cópias de rotinas.	Novas estratégias, observadores e buscas podem aproveitar os contratos e mecanismos introduzidos.
Manutenção	Correções em operações repetidas precisavam ser aplicadas em mais de um lugar.	Ajustes nas rotinas compartilhadas passam a ser concentrados nas implementações comuns.
Reutilização	O reaproveitamento de algoritmos e mecanismos de travessia era limitado pela organização existente.	O esqueleto de BFS, a iteração do tabuleiro e os contratos de estratégias e observadores podem atender a novas implementações.

As refatorações introduziram mais classes e interfaces, aumentando o esforço inicial de compreensão do código. Esse custo deve ser considerado em relação às possíveis extensões do sistema: as abstrações oferecem maior benefício quando novos comportamentos e operações aproveitam os pontos de extensão criados. Neste trabalho, essas possibilidades foram analisadas pela estrutura do código, sem medir o esforço necessário para implementar futuras funcionalidades. Assim, as alterações oferecem caminhos para facilitar a evolução do sistema, mas não permitem quantificar a economia de trabalho em extensões posteriores.

5. Conclusão

O trabalho analisou e refatorou componentes do JPacman Framework por meio dos padrões Strategy, Observer, Template Method e Iterator, com uma intervenção complementar utilizando Visitor. As alterações trataram problemas concretos de duplicação, dependências rígidas e ausência de abstrações, estabelecendo pontos de extensão e mecanismos compartilhados para a evolução do sistema.

A comparação das estruturas mostrou possibilidades de reutilizar algoritmos e operações com menor necessidade de repetir código ou modificar componentes centrais. A execução dos testes automatizados e as verificações manuais forneceram evidências de preservação dos comportamentos exercitados, dentro dos limites da validação realizada.

O principal desafio técnico foi separar as operações compartilhadas das variações específicas sem alterar as regras existentes. Esse cuidado foi especialmente relevante nas buscas, na ordem de travessia do tabuleiro e na resolução de colisões.

A experiência reforçou que a aplicação de padrões deve partir dos problemas do sistema e considerar os custos das abstrações introduzidas. Sua contribuição para o reuso depende de oferecer contratos e responsabilidades que possam ser aproveitados em novas implementações. Neste trabalho, as refatorações estabeleceram pontos de extensão que podem facilitar a introdução de novos comportamentos e operações, aproveitando estruturas existentes e preservando os comportamentos verificados.

6. Referências e ferramentas utilizadas

6.1. Referências

REFACTORING GURU. Padrões de projeto. Disponível em: https://refactoring.guru/pt-br/design-patterns. Acesso em: 12 out. 2026. Utilizado para estudar os padrões GoF, compreender suas aplicações e preparar a apresentação.

SERG — DELFT UNIVERSITY OF TECHNOLOGY. JPacman Framework. Disponível em: https://github.com/SERG-Delft/jpacman-framework. Acesso em: 12 out. 2026. Projeto utilizado como base para a análise e as refatorações.

6.2. Uso de ferramentas de inteligência artificial

As ferramentas de inteligência artificial foram utilizadas como apoio nas seguintes atividades:

- Gemini 3.8 Flash: apoio à análise do código e à identificação de possíveis problemas de projeto e oportunidades de refatoração.

- Claude Sonnet 4.6, Big Pickle e GPT-6.1 Sol: apoio à implementação das refatorações.
- Claude Sonnet 5.5 e GPT-6.1 Sol: apoio à redação, organização e revisão do relatório.